TECHNICAL DESIGN DOCUMENT

GridEye: Edge-AI Integrated Real-Time Power Monitoring and Differential Theft Detection

Document Version: 1.0
Date: October 2026
Project Type: Academic Engineering Prototype
Technologies: ESP32, PZEM-004T v3, Random Forest, IoT, ThingSpeak

---

1. Introduction

GridEye is an experimental electrical monitoring system that combines electrical measurement, embedded machine learning, and Internet of Things (IoT) technology to classify configured electrical operating conditions.

The system uses an ESP32 microcontroller and two PZEM-004T v3 energy measurement modules to collect electrical parameters from the supply-side and consumer-side test setups. The measured values are processed to calculate differential features, which are passed to an embedded Random Forest classifier.

The model classifies the measurements into four configured operating conditions. The system can also upload electrical measurements to ThingSpeak when Wi-Fi connectivity and the required configuration are available.

This project explores a low-cost approach to electrical monitoring using embedded artificial intelligence.

2. Problem Statement

Electricity theft and power losses can affect the efficiency and revenue of electrical distribution systems. Monitoring electrical parameters at different points can help identify unusual differences that require further investigation.

Conventional monitoring approaches may depend on manual inspection or predefined thresholds. GridEye explores the use of differential electrical measurements and machine learning to classify selected operating conditions.

The prototype is designed for experimental testing. Its classification results are not conclusive proof of electricity theft in a real distribution network.

3. Project Objectives

- Monitor voltage, current, power, and power factor at two measurement points.
- Compare supply-side and consumer-side electrical parameters.
- Calculate differential features from measured values.
- Implement Random Forest inference directly on the ESP32.
- Classify four configured operating conditions.
- Support remote data monitoring through ThingSpeak.
- Enable local classification without requiring continuous internet connectivity.
- Establish a foundation for future improvements in electrical monitoring.

4. Scope of the Project

The current implementation consists of an ESP32, two PZEM-004T v3 energy measurement modules, and two 9 W bulbs used as experimental loads.

The ESP32 reads the measurements, calculates the required features, and runs the embedded Random Forest model. When configured, it can upload monitoring data to ThingSpeak.

The current prototype does not include a deployed distribution-network installation, Raspberry Pi-based processing, or GPS-based feeder tracking. These may be considered future enhancements.

5. System Architecture

5.1 Architecture Overview

The system consists of five main functional stages:

1. Electrical parameter measurement.
2. Data acquisition through the ESP32.
3. Differential feature calculation.
4. Embedded Random Forest classification.
5. Local output and optional cloud monitoring.

5.2 Architecture Diagram

```
flowchart TD
    A[Supply-Side Test Setup] --> B[PZEM Sensor 1]
    C[Consumer-Side Test Setup] --> D[PZEM Sensor 2]
    B --> E[ESP32 Microcontroller]
    D --> E
    E --> F[Electrical Feature Calculation]
    F --> G[Embedded Random Forest Model]
    G --> H[Operating Condition Classification]
    H --> I[Serial Monitor / Local Output]
    E -. Wi-Fi when available .-> J[ThingSpeak Cloud]
```

Figure 1: GridEye system architecture

5.3 Working Principle

The two PZEM sensors measure electrical parameters at their respective points in the experimental setup. The ESP32 collects these measurements and calculates the differential features.

The resulting feature vector is supplied to the embedded Random Forest model. The model evaluates the input using its trained decision trees and returns the predicted class.

The classification can be viewed through the local output. If Wi-Fi and ThingSpeak settings are configured, measurement data can also be uploaded for remote monitoring.

6. Hardware Design

6.1 ESP32 Development Board

The ESP32 acts as the central processing unit. It reads the sensor measurements, performs feature calculations, executes the embedded machine learning model, and handles optional Wi-Fi communication.

6.2 PZEM-004T v3 Energy Measurement Modules

Two PZEM-004T v3 modules are used to measure electrical parameters from the two sides of the experimental setup.

The measured parameters include:

- Voltage
- Current
- Active power
- Power factor

The firmware must use the correct serial interface and pin assignments for the actual hardware connections.

6.3 Experimental Loads

Two 9 W bulbs are used to create controlled load conditions for testing the prototype's classification logic.

The bulbs allow different configured operating conditions to be demonstrated under controlled circumstances.

6.4 Hardware Components

Component	Function
ESP32 development board	Data processing and embedded inference
PZEM-004T v3 Sensor 1	Supply-side electrical measurement

PZEM-004T v3 Sensor 2	Consumer-side electrical measurement
Two 9 W bulbs	Experimental load conditions
Computer	Programming, training, and serial monitoring

6.5 Electrical Safety

The system involves AC electrical measurements. All mains wiring and modifications must be carried out by a qualified person using suitable protection, insulation, and appropriately rated components. Power must be disconnected before wiring changes. Exposed live conductors must never be handled.

7. Software Design

7.1 ESP32 Firmware

The firmware is developed using C++ in the Arduino IDE.

The main firmware operations are:

1. Initialize the ESP32 and sensor communication interfaces.
2. Read the electrical measurements from both PZEM sensors.
3. Prepare the sensor values for processing.
4. Calculate the differential features.
5. Construct the model input vector in the required feature order.
6. Run the embedded Random Forest prediction function.
7. obtain the predicted class and display the result.
8. Upload configured measurements to ThingSpeak when connectivity is available.

7.2 Embedded Machine Learning Model

The trained Random Forest model is represented by a C/C++ header included in the ESP32 firmware.

The header contains the model's decision-tree information and prediction logic. The ESP32 uses this representation to perform inference without requiring a Python runtime.

The model header, firmware, feature order, and class order must remain consistent.

Retraining the Python model does not automatically update the embedded header. If a new model is trained, its embedded representation must be generated and the firmware recompiled before deployment.

8. Dataset and Feature Engineering

8.1 Dataset

The training dataset is stored in:

`dataset/GridEye_combined_clean_dataset.csv`

The recorded dataset contains 275 samples distributed across four classes.

Class	Number of Samples
Normal Operation (Consumer Only)	100
Idle / Both OFF	72
Direct Theft (Bypass)	60
Partial Theft (Both Loads Active)	43
Total	**275**

8.2 Input Features

The Random Forest model uses 11 input features.

No.	Feature	Description
1	V1	Supply-side voltage
2	I1	Supply-side current
3	P1	Supply-side power
4	PF1	Supply-side power factor
5	V2	Consumer-side voltage
6	I2	Consumer-side current
7	P2	Consumer-side power
8	PF2	Consumer-side power factor
9	Power_Diff	Difference between supply-side and consumer-side power
10	Current_Diff	Difference between supply-side and consumer-side current
11	Voltage_Diff	Absolute difference between the two voltage measurements

The differential features are calculated from the corresponding sensor readings before classification.

The order and definitions of these features must match the training notebook and the embedded model header.

9. Random Forest Classification

9.1 Model Description

Random Forest is a supervised machine learning algorithm that combines the predictions of multiple decision trees to produce a classification.

Each decision tree evaluates the input features using learned conditions. The individual tree predictions are combined to determine the final output class.

GridEye uses a Random Forest model trained using the project's labelled dataset and embedded into the ESP32 firmware.

9.2 Training Methodology

The training notebook uses the following configuration:

- Algorithm: Random Forest Classifier
- Number of trees: 100
- Random seed: 42
- Training and test split: 80/20
- Stratification: Enabled
- Cross-validation: Five-fold cross-validation on the training split

The notebook is stored in the `ml_train/` directory.

9.3 Classification Output

The model predicts one of the following four configured classes:

1. **Direct Theft (Bypass):** Represents the configured bypass condition in the dataset.
2. **Idle / Both OFF:** Represents the condition where both test loads are switched off.
3. **Normal Operation (Consumer Only):** Represents the configured consumer-only operating condition.
4. **Partial Theft (Both Loads Active):** Represents the configured condition where both test loads are active.

These labels correspond to the experimental conditions represented in the dataset. Real distribution networks may exhibit conditions that are not represented by the training data.

10. Operating Workflow

The system follows this workflow:

1. The ESP32 initializes the sensor communication interfaces.
2. Both PZEM sensors measure electrical parameters.
3. The ESP32 reads the available measurements.
4. The firmware calculates the differential features.
5. The values are arranged in the expected model input order.
6. The embedded Random Forest model processes the feature vector.
7. The predicted class is displayed through the local output.
8. The firmware attempts cloud upload when ThingSpeak is configured and network connectivity is available.
9. The monitoring and classification cycle continues.

Figure 2: GridEye operating workflow

11. ThingSpeak Integration

ThingSpeak is used as an optional cloud platform for monitoring uploaded electrical measurements.

The configured data fields are:

Field	Description
V1	Supply-side voltage
I1	Supply-side current
P1	Supply-side power
PF1	Supply-side power factor
V2	Consumer-side voltage
I2	Consumer-side current
P2	Consumer-side power
PF2	Consumer-side power factor
Power_Diff	Power difference
Current_Diff	Current difference
Voltage_Diff	Voltage difference

The ESP32 uses Wi-Fi to communicate with ThingSpeak. Cloud upload requires valid network settings, channel configuration, and API credentials.

Local classification is designed to operate independently of continuous internet access. Cloud data transmission may be interrupted when connectivity is unavailable.

Actual upload frequency and field mapping must match the firmware and the ThingSpeak channel configuration.

12. Testing and Validation

Testing is intended to verify the measurement process, feature calculation, model output, and optional cloud communication.

12.1 Sample Test Cases

Test Case	Test Condition	Expected Classification
1	Both test loads switched off	Idle / Both OFF
2	Consumer-only load operating in the configured normal setup	Normal Operation (Consumer Only)
3	Configured bypass test condition	Direct Theft (Bypass)
4	Both test loads operating in the configured partial-theft setup	Partial Theft (Both Loads Active)
5	Wi-Fi unavailable	Local measurement and inference should continue

6	Valid Wi-Fi and ThingSpeak configuration	Configured measurements should upload

These are expected results, not a claim that every test has passed. Actual measurements and predicted outputs should be recorded during hardware validation.

12.2 Model Evaluation

The training notebook performs a train-test split and cross-validation. Any accuracy values reported in a final project presentation or paper should be reproduced from the current notebook and supported by the evaluation procedure.

Because the dataset is limited, further validation using independent measurements is recommended. Similar or repeated samples can lead to overly optimistic evaluation results.

13. Limitations

- The prototype is a small-scale experimental setup rather than a deployed electricity distribution monitoring system.
- The dataset contains a limited number of samples representing configured test conditions.
- The model may not generalize to real-world operating conditions that differ from the training data.
- Sensor errors, voltage variations, load changes, and technical losses can affect differential measurements.
- A predicted class does not independently establish that electricity theft has occurred.
- Cloud monitoring depends on Wi-Fi connectivity and correct API configuration.
- The current implementation does not include all proposed feeder-level, GPS, or Raspberry Pi features.

14. Future Enhancements

Potential improvements include:

- Collecting a larger dataset with a wider range of controlled electrical conditions.
- Testing the model on independent data not used during training.
- Investigating duplicate samples and possible data leakage.
- Adding sensor fault detection and measurement-quality checks.
- Developing alerts for sustained abnormal measurements.
- Exploring feeder-level monitoring and location-aware reporting.
- Investigating LoRa or other long-range communication technologies for areas with limited connectivity.
- Validating the design under realistic distribution-system conditions with appropriate electrical safety procedures.

These enhancements require further design, implementation, and validation before they can be considered completed features.

15. Repository Structure

The project repository contains the following main components:

```
GridEye/
├── dataset/
│   └── GridEye_combined_clean_dataset.csv
├── GridEye_ESP32_Retrained_RandomForest/
│   ├── ESP32 firmware (.ino)
│   └── Embedded model header file(s)
├── ml_train/
│   └── Jupyter Notebook
├── docs/
│   └── Technical_Design_Document.pdf
├── README.md
├── LICENSE
└── .gitignore
```

The names shown descriptively in parentheses must be replaced by the actual filenames if this structure is copied literally. Existing source filenames should remain unchanged.

16. Conclusion

GridEye demonstrates an experimental integration of electrical measurement, differential feature engineering, embedded Random Forest classification, and optional IoT-based monitoring.

The prototype provides a platform for exploring how machine learning can classify configured electrical operating conditions using measurements from two points. Further dataset development, independent validation, safety review, and testing under realistic operating conditions are necessary before considering broader deployment.

17. License

GridEye is distributed under the MIT License. Refer to the repository's `LICENSE` file for the complete license terms.

Project Title: GridEye: Edge-AI Integrated Real-Time Power Monitoring and Differential Theft Detection